Universidad Nacional de Córdoba
Facultad de Ciencias Exactas, Físicas y Naturales

FCEFyN

Procesamiento Digital de Señales

Ingeniería en Computación

Laboratorio 3

FFT con CMSIS-DSP en MCXN947

FRDM-MCXN947 · MCUXpresso IDE · CMSIS-DSP · MATLAB

Integrante	**Matrícula**	**Correo**
Costamagna, Matias Javier	44.512.437	matias.costamagna@mi.unc.edu.ar
Moreyra, Julián	45.214.537	julian.moreyra@mi.unc.edu.ar
Rivera, Mariano	45.432.921	luismarianorivera.25@mi.unc.edu.ar
Rojas, Facundo Nicolás	44.629.222	facundo.rojas.222@mi.unc.edu.ar

Docentes: Rossi, Roberto Raúl · Parlanti, Gustavo Fernando · Molina, Germán Alcides · Ferreyra, Pablo Alejandro

Fecha de entrega: 5 de noviembre de 2026

Índice

1. Introducción **3**
1.1. Requerimientos . . . 3

2. Configuración del ambiente **5**
2.1. Herramientas utilizadas . . . 5
2.2. Creación del proyecto . . . 5
2.3. Relojes y pines . . . 5
2.4. Cambios en los periféricos . . . 6

3. La Transformada Rápida de Fourier **9**
3.1. Transformada discreta de Fourier . . . 9
3.2. Algoritmo de la FFT . . . 9
3.3. Implementación en CMSIS-DSP . . . 10
3.4. Interpretación de la amplitud . . . 10
3.5. Fuga espectral y ventana . . . 11

4. Diseño de la aplicación **12**
4.1. Arquitectura general . . . 12
4.2. Velocidades de muestreo . . . 12
4.3. Resolución y duración de la ventana . . . 13
4.4. Máquina de estados . . . 13
4.5. Procesamiento por bloques . . . 14
4.6. Formato de los datos . . . 15
4.7. Protocolo de comunicación . . . 15
4.8. Restricción de tiempo . . . 16
4.9. Uso de memoria . . . 17

5. Implementación **18**
5.1. Constantes y variables globales . . . 18
5.2. Programa principal . . . 18
5.3. Rutina del ADC . . . 19
5.4. Procesamiento del bloque . . . 20
5.5. Transmisión . . . 21
5.6. Rutinas de las teclas . . . 22
5.7. Touchpad . . . 22
5.8. Aplicación de MATLAB . . . 22

6. Verificación **24**

6.1. Procedimiento 24
6.2. Verificación por simulación 24
6.3. Transmisión y modo *bypass* 25
6.4. Espectro al variar el tamaño de la ventana 25
6.5. Espectro al variar la frecuencia de muestreo 26
6.6. Aliasing y armónicos 26
6.7. Valores medidos 27
6.8. Cumplimiento de los requerimientos 28

7. Conclusiones **29**
7.1. Efecto de la frecuencia de muestreo y del tamaño de la ventana 29
7.2. Decisiones de diseño 29
7.3. Mejoras posibles 30

A. Código fuente **31**

1 Introducción

Laboratorio 3 — Transformada Rápida de Fourier

Objetivo. Realizar la implementación de la Transformada Rápida de Fourier (FFT) de 512 muestras.
Descripción. Realizar un programa aplicativo que sea capaz de aplicar la FFT a un buffer de memoria de diferente tamaño (por ejemplo, 512, 1024 y 2048 muestras), que es adquirido con el Laboratorio 1 teniendo en cuenta además los cambios de velocidades de muestreo. Con una de las teclas de la placa de evaluación se habilitará la aplicación de la FFT o se hará *bypass* de la misma a un buffer de salida distinto del buffer de entrada. Transmitir el buffer (resultado de la FFT) por el puerto serie (SCI) y graficar en la PC con alguna aplicación (por ejemplo, una GUI de MATLAB) el espectro obtenido. Sacar conclusiones respecto de la implementación de la FFT cuando cambia la frecuencia de muestreo o el tamaño de la ventana de muestras.

1.1 Requerimientos

De la consigna se desprenden los siguientes requerimientos, que se usan como guía para el diseño y como lista de verificación en la sección 6:

R1 Adquirir la señal con la aplicación del Laboratorio 1, con sus cinco velocidades de muestreo (8 kS/s, 16 kS/s, 22 kS/s, 44 kS/s y 48 kS/s) seleccionables durante la ejecución.

R2 Guardar las muestras en un buffer de entrada de tamaño variable: 512, 1024 o 2048 muestras.

R3 Aplicar la FFT al buffer de entrada.

R4 Con una tecla, habilitar la FFT o hacer *bypass* hacia un buffer de salida distinto del de entrada.

R5 Transmitir el buffer de salida por el puerto serie.

R6 Graficar en la PC, con una aplicación de MATLAB, el espectro recibido.

R7 Sacar conclusiones sobre el efecto de la frecuencia de muestreo y del tamaño de la ventana en la FFT.

Decisiones de implementación respecto de la consigna

- El tamaño de la ventana se elige durante la ejecución, sin recompilar: el touchpad capacitivo E1 recorre 512, 1024 y 2048 muestras. SW3 conserva la selección circular de la frecuencia de muestreo de los laboratorios anteriores y SW2 alterna entre FFT y *bypass*, como pide la consigna.
- El procesamiento es *por bloques*: la rutina del ADC llena el buffer de entrada y el lazo principal calcula la FFT y transmite el resultado. Mientras tanto la adquisición sigue disparándose por hardware, pero las muestras se descartan hasta que el bloque termina de enviarse. Cada espectro corresponde entonces a una ventana de N muestras consecutivas, y entre dos espectros hay un hueco. La sección 4.8 muestra que con el puerto serie a 115 200 bit/s no es posible transmitir espectros en forma continua.
- La FFT se calcula en punto flotante de simple precisión con `arm_cfft_f32()` de CMSIS-DSP, aprovechando la FPU del Cortex-M33 (sección 4.6).
- En *bypass* el buffer de salida contiene las muestras de la ventana, sin transformar, y la GUI las grafica en el dominio del tiempo. Así el mismo enlace sirve para ver la señal y su espectro.

- El TP3 no usa el DAC: la salida es el puerto serie. El LED RGB conserva el color de cada frecuencia de muestreo; el modo, el tamaño de la ventana y la f_s de cada bloque viajan en la cabecera de la trama y la GUI los muestra en el título del gráfico.

2 Configuración del ambiente

2.1 Herramientas utilizadas

Tabla 1: Hardware y software del entorno de trabajo.

Elemento	**Versión / modelo**
Placa de evaluación	FRDM-MCXN947 (Cortex-M33 con FPU, núcleo `cm33_core0`)
IDE	MCUXpresso IDE v25.6.136
SDK	`SDK_26.06.00_FRDM-MCXN947`
Herramientas de config.	MCUXpresso Config Tools (Clocks, Pins, Peripherals)
Proyecto base	Proyecto del Laboratorio 2 (`TP2`)
Drivers	SDK: `fsl_lpuart`, `fsl_tsi_v6`, `fsl_lpadc`, `fsl_ctimer`
Biblioteca DSP	CMSIS-DSP (`arm_cfft_f32`)
Visualización	MATLAB (función `serialport`, R2019b o posterior)
Enlace serie	Puerto COM virtual del MCU-Link integrado (USB)
Generador de señales	
Osciloscopio	

2.2 Creación del proyecto

El proyecto `TP3` parte del proyecto del Laboratorio 2: conserva la cadena CTIMER0 → INPUTMUX → LPADC0, la selección circular de la frecuencia de muestreo con SW3, el color del LED según la f_s y la lectura del touchpad sin interrupciones. Sobre esa base:

1. Se quitó el DAC0 y el filtrado muestra a muestra: la rutina del ADC sólo guarda muestras, y el procesamiento pasa al lazo principal.
2. Se mantuvo el código fuente de CMSIS-DSP (carpeta `CMSIS/DSP`), que además de los filtros incluye las funciones de transformadas (`TransformFunctions`) y las tablas precalculadas (`CommonTables`): `arm_cfft_f32()`, `arm_bitreversal_32()` y las instancias constantes `arm_cfft_sR_f32_len512`, `_len1024` y `_len2048` de `arm_const_structs.h`.
3. Se configuró en Config Tools la instancia `LP_FLEXCOMM4` como LPUART (sección 2.4), que es la que la placa conecta al puerto COM virtual del MCU-Link.
4. Se agregó el componente `tsi_v6` del SDK (driver `fsl_tsi_v6`) para leer el touchpad, igual que en el Laboratorio 2.
5. La lógica de la aplicación está en `source/TP3.c` y la GUI y las herramientas de prueba de MATLAB en `TP3/matlab/` (sección 5.8).

2.3 Relojes y pines

Los relojes son los del Laboratorio 2: PLL0 a 150 MHz para el núcleo y el CTIMER0, y FRO_HF de 48 MHz para el ADC0. Se agrega el FRO de 12 MHz como reloj funcional del LP_FLEXCOMM4 (la UART) y del TSI. La tabla 2 resume la asignación de pines.

Tabla 2: Asignación de pines. Respecto del Laboratorio 2 desaparece la salida del DAC y se agrega la UART.

Función	Pin	Periférico	Conector / observación
Entrada analógica	`ADC0_A0`	LPADC0 canal A0	`J4[2]` (Arduino A0)
Disparo del ADC	`P0_19`	CT0_MAT3	`J2[13]`, punto de prueba
LED rojo / verde / azul	`P0_10`, `P0_27`, `P1_2`	GPIO0, GPIO1	Activos en bajo
Tecla SW3 (velocidad)	`P0_6`	GPIO0, canal IRQ 0	Flanco descendente, pull-up interno
Tecla SW2 (FFT / *bypass*)	`P0_23`	GPIO0, canal IRQ 1	Flanco descendente
Touchpad E1 (tamaño N)	`P1_3`	TSI0, canal 3	Self-cap, disparo por software
UART RXD / TXD	`P1_8`, `P1_9`	LP_FLEXCOMM4	COM virtual del MCU-Link

[Verificar que el TP3 tenga en Config Tools `P1_3` como `TSI0_CH3` y el reloj del TSI desde FRO_12M, como en el TP2: en el estado actual del repositorio faltan, y sin ellos el touchpad queda deshabilitado y N fijo en 512.]

Captura pendiente: **Config Tools — Pins**
guardar como `img/pins-tsi-tp3.png`

Figura 1: Config Tools — Pins. `P1_3` asignado a `TSI0_CH3` (touchpad E1), con el buffer de entrada digital deshabilitado.

Captura pendiente: **Config Tools — Clocks**
guardar como `img/clocks-tsi-tp3.png`

Figura 2: Config Tools — Clocks. Reloj del TSI tomado del FRO de 12 MHz.

2.4 Cambios en los periféricos

La tabla 3 resume lo que cambió respecto del Laboratorio 2. El resto de la configuración (CTIMER0 en modo toggle sobre el match 3, disparo del ADC por INPUTMUX, LPADC0 en 16 bit con referencia `kLPADC_ReferenceVoltageAlt3`, interrupciones de las teclas en las líneas 0 y 1 del GPIO0) se mantiene.

Tabla 3: Diferencias de configuración respecto del Laboratorio 2.

Elemento	Laboratorio 2	Laboratorio 3
DAC0	`init_dac()`, una muestra por interrupción	No se usa
VREF	Encendido por `init_dac()` (`SPC0->ACTIVE_CFG1`)	Encendido explícito con `SPC_EnableActiveModeAnalogModules()` antes de los periféricos
LP_FLEXCOMM4	Consola de depuración (`PRINTF`)	LPUART de Config Tools, 115 200 bit/s, 8N1, sólo transmisión, por *polling*
Tecla SW2	Selección de filtro	FFT / *bypass*
Touchpad E1	Habilita el filtro	Elige N: 512 → 1024 → 2048 → 512
LED RGB	Color según la f_s; parpadeo según el filtro	Color según la f_s
Procesamiento	Muestra a muestra, en la rutina del ADC	Por bloques de N muestras, en el lazo principal
Buffers	`buffer_in[512]` y `buffer_out[512]` (q15, circulares)	`buffer_in[2048]` (q15) y `buffer_out[4096]` (float32, complejo intercalado), lineales
CMSIS-DSP	`arm_fir_q15()`	`arm_cfft_f32()` y tablas de `arm_const_structs.h`
Lazo principal	Touchpad y LED	Touchpad, FFT y transmisión

Captura pendiente: **Config Tools — LP_FLEXCOMM4**
guardar como `img/peripherals-lpuart-tp3.png`

Figura 3: Config Tools — LP_FLEXCOMM4. Instancia LPUART en modo *polling*: 115 200 bit/s, 8 bits de datos, sin paridad, 1 bit de parada, sólo transmisión.

2.4.1 Orden de inicialización y referencia del ADC

El LPADC0 usa la referencia `kLPADC_ReferenceVoltageAlt3`, que proviene del módulo VREF. En el Laboratorio 2 el VREF quedaba encendido como efecto lateral de `init_dac()`, que escribía `SPC0->ACTIVE_CFG1 |= 0x11` para alimentar el DAC0 y el VREF. Al quitar el DAC en el TP3, el VREF quedó apagado y el ADC dejó de entregar bloques válidos: no salía ninguna trama por la UART. Por eso `main()` enciende el VREF en forma explícita con

```
SPC_EnableActiveModeAnalogModules(SPC0, kSPC_controlVref);
```

antes de llamar a `BOARD_InitBootPeripherals()`, que calibra el ADC y arranca el CTIMER0. Así la calibración y la primera conversión ya se hacen con la referencia estable.

2.4.2 Prioridades de interrupción

Igual que en los laboratorios anteriores, `ADC0_IRQn`, `GPIO00_IRQn` y `GPIO01_IRQn` quedan con la prioridad por defecto (0), así que ninguna desaloja a otra y las variables que comparten no pueden cambiar en medio de una rutina. El SysTick, que sólo cuenta milisegundos para el touchpad, queda con la prioridad mínima (`SysTick_Config()` la fija así).

A diferencia del Laboratorio 2, el trabajo pesado (la FFT y la transmisión) no está en ninguna rutina de interrupción sino en el lazo principal, que corre en modo *thread* y puede ser interrum-

pido en cualquier momento. La rutina del ADC, que es corta, sigue atendiendo cada conversión durante los segundos que dura una transmisión, y por eso la FIFO del ADC nunca se desborda aunque las muestras no se guarden.

3 La Transformada Rápida de Fourier

3.1 Transformada discreta de Fourier

La transformada discreta de Fourier (DFT) de N puntos de una secuencia $x[n]$, $n = 0, \ldots, N-1$, es

$$X[k] = \sum_{n=0}^{N-1} x[n]\, W_N^{kn}, \qquad W_N = e^{-j2\pi/N}, \qquad k = 0, \ldots, N-1. \tag{1}$$

Si $x[n]$ son muestras tomadas a f_s, el coeficiente $X[k]$ corresponde a la frecuencia $f_k = k\, f_s/N$, así que la DFT evalúa el espectro en una grilla de frecuencias equiespaciadas con separación

$$\Delta f = \frac{f_s}{N} = \frac{1}{T_v}, \qquad T_v = \frac{N}{f_s}, \tag{2}$$

donde T_v es la duración de la ventana analizada. La ecuación (2) es la que gobierna todo el trabajo: la resolución en frecuencia es la inversa de la duración de la ventana, y no depende de f_s y de N por separado sino de su cociente.

Cuando la entrada es real, como la del ADC, los coeficientes cumplen $X[N-k] = X^*[k]$: la mitad superior del espectro es la imagen conjugada de la inferior y la información está en los $N/2+1$ coeficientes que van de 0 Hz a $f_s/2$. Los $k > N/2$ corresponden a frecuencias negativas.

3.2 Algoritmo de la FFT

Calcular la ecuación (1) en forma directa requiere N productos complejos por coeficiente, es decir, N^2 en total. La FFT obtiene el mismo resultado aprovechando la periodicidad y la simetría de W_N^{kn}. En la versión de diezmado en el tiempo (radix-2) la secuencia se separa en sus muestras pares e impares, $g[m] = x[2m]$ y $h[m] = x[2m+1]$, cuyas DFT de $N/2$ puntos son $G[k]$ y $H[k]$. Entonces

$$X[k] = G[k] + W_N^k H[k], \qquad X[k+N/2] = G[k] - W_N^k H[k], \qquad k = 0, \ldots, N/2-1, \tag{3}$$

porque $W_N^{k+N/2} = -W_N^k$. Cada par de la ecuación (3) es una *mariposa*: un producto complejo por el factor de giro (*twiddle*) W_N^k y dos sumas complejas. Aplicando la división recursivamente hasta DFT de 2 puntos quedan $\log_2 N$ etapas de $N/2$ mariposas cada una, con un total de $(N/2)\log_2 N$ productos complejos. El precio es el orden: si el cálculo se hace en el lugar (*in place*), una de las dos secuencias, la entrada o la salida, queda en orden de *bits invertidos* (el índice n escrito en binario y leído al revés) y hay que reordenarla.

Tabla 4: Productos complejos de la DFT directa y de la FFT radix-2 para los tres tamaños de ventana.

N	Etapas	DFT: N^2	FFT: $(N/2)\log_2 N$	Relación
512	9	262 144	2 304	113,8
1 024	10	1 048 576	5 120	204,8
2 048	11	4 194 304	11 264	372,4

La ventaja crece con N: al duplicar la ventana la DFT directa cuesta cuatro veces más y la FFT apenas algo más del doble. Para $N = 2048$ la DFT directa necesitaría más de cuatro millones de productos complejos, que a 150 MHz y con varios ciclos por producto serían más de 100 ms; la FFT lo resuelve con unos 11 000.

3.3 Implementación en CMSIS-DSP

`arm_cfft_f32()` calcula la FFT compleja de N puntos en punto flotante de simple precisión. Recibe un arreglo de $2N$ valores `float32_t` con la parte real y la imaginaria intercaladas, $\{\mathrm{Re}\,x[0], \mathrm{Im}\,x[0], \mathrm{Re}\,x[1], \dots\}$, y devuelve el resultado en el mismo arreglo. Sus parámetros son:

- `S`: puntero a una instancia `arm_cfft_instance_f32` con la longitud N, la tabla de factores de giro y la tabla de reordenamiento. CMSIS-DSP las trae precalculadas en flash como constantes (`arm_cfft_sR_f32_len512`, ...), así que no hay que inicializar nada en tiempo de ejecución.
- `ifftFlag`: 0 para la transformada directa, 1 para la inversa.
- `bitReverseFlag`: 1 para reordenar la salida y obtener los coeficientes en orden natural ($X[0], X[1], \dots$).

Internamente no usa mariposas radix-2 sino radix-8, que combinan ocho entradas por mariposa y reducen la cantidad de etapas a $\log_8 N$ y la de multiplicaciones por factores de giro. Como $512 = 8^3$, la FFT de 512 puntos se resuelve sólo con radix-8 (`arm_radix8_butterfly_f32()`). Para 1024 y 2048 agrega primero una etapa radix-2 o radix-4 que divide el problema en dos o cuatro FFT de 512 (`arm_cfft_radix8by2_f32()` y `arm_cfft_radix8by4_f32()`). Al final, si `bitReverseFlag` vale 1, `arm_bitreversal_32()` intercambia los pares de posiciones indicados en la tabla de reordenamiento. La tabla 5 muestra la memoria que ocupan esas tablas.

Tabla 5: Tablas precalculadas de CMSIS-DSP para cada tamaño (en flash).

N	**Factores de giro**	**Bytes**	**Reordenamiento**	**Bytes**
512	`twiddleCoef_512`, 1 024 float	4 096	`armBitRevIndexTable512`, 448 `uint16`	896
1024	`twiddleCoef_1024`, 2 048 float	8 192	`armBitRevIndexTable1024`, 1 800 `uint16`	3 600
2048	`twiddleCoef_2048`, 4 096 float	16 384	`armBitRevIndexTable2048`, 3 808 `uint16`	7 616

CMSIS-DSP también ofrece `arm_rfft_fast_f32()`, que aprovecha que la entrada es real: calcula una FFT compleja de $N/2$ puntos y la separa en los $N/2 + 1$ coeficientes no redundantes, con aproximadamente la mitad de operaciones y de memoria. Se eligió la transformada compleja porque su salida es directamente la DFT completa de la ecuación (1), sin el formato empaquetado de la real, y porque con una entrada real basta con poner en cero la parte imaginaria. La sección 4.8 muestra que el costo de cálculo es despreciable frente al de la transmisión, así que el ahorro no cambia el comportamiento del sistema.

3.4 Interpretación de la amplitud

`arm_cfft_f32()` no normaliza la transformada directa. Para una senoidal de amplitud A (en las unidades normalizadas de la sección 4.6) cuya frecuencia coincide exactamente con un coeficiente k_0, la ecuación (1) da $|X[k_0]| = |X[N - k_0]| = A\,N/2$: la amplitud se reparte entre la frecuencia positiva y su imagen negativa, y cada una crece con N. Para que el gráfico no dependa del tamaño de la ventana, la GUI dibuja el espectro de un solo lado,

$$A[0] = \frac{|X[0]|}{N}, \qquad A[k] = \frac{2\,|X[k]|}{N}, \quad 1 \le k \le \frac{N}{2} - 1, \qquad A[N/2] = \frac{|X[N/2]|}{N}, \tag{4}$$

con lo que una senoidal de amplitud A aparece con altura A y la componente continua con su valor medio. Se calcula en la PC y no en la placa para enviar el resultado exacto de `arm_cfft_f32()`.

3.5 Fuga espectral y ventana

La DFT trata la ventana de N muestras como un período de una señal periódica. Si la señal tiene un número entero de ciclos en la ventana, su frecuencia cae exactamente sobre un coeficiente y el espectro tiene un único valor no nulo. Si no, la extensión periódica tiene un salto en los bordes y la energía se reparte entre todos los coeficientes: es la *fuga espectral* (*leakage*). Equivale a multiplicar la señal por una ventana rectangular, cuya transformada es un núcleo de Dirichlet con lóbulo principal de ancho $2\,\Delta f$ y lóbulos laterales que empiezan en $-13{,}3\,\text{dB}$ y decaen lentamente.

La fuga tiene dos consecuencias prácticas:

- **Error de amplitud** (*scalloping*): si la frecuencia cae entre dos coeficientes, el pico es menor que A. En el peor caso, a medio coeficiente, la pérdida es de 3,92 dB (A se ve como $0{,}64\,A$).
- **Enmascaramiento**: los lóbulos laterales de un tono fuerte pueden tapar a un tono débil cercano.

El script `explorar_fft_tp3.m` lo ilustra con $f_s = 8\,\text{kS/s}$ y $N = 512$ ($\Delta f = 15{,}625\,\text{Hz}$). Una senoidal de 1 000 Hz tiene exactamente 64 ciclos en la ventana: cae en el coeficiente 64 y la amplitud se lee sin error. Con 1 005 Hz hay 64,32 ciclos: el pico queda en 1 000 Hz con amplitud $0{,}84\,A$, el coeficiente vecino (1 015,6 Hz) recibe $0{,}40\,A$ y los demás no se anulan (tabla 6).

Tabla 6: Amplitud de un solo lado, ecuación (4), alrededor del pico para una senoidal de $A = 0{,}5$ cuantizada a q15, con $f_s = 8\,\text{kS/s}$ y $N = 512$ (calculada con la DFT de la ecuación (1)).

k	f_k **[Hz]**	$A[k]$ **con** $f_0 = 1\,000\,\text{Hz}$	$A[k]$ **con** $f_0 = 1\,005\,\text{Hz}$
62	968,75	0,000 0	0,057 5
63	984,375	0,000 0	0,101 4
64	1 000	0,500 0	0,419 5
65	1 015,625	0,000 0	0,198 0
66	1 031,25	0,000 0	0,080 4

La implementación no aplica ninguna ventana además de la rectangular, para que el espectro enviado sea exactamente la FFT del buffer. Una ventana de Hann (`arm_hanning_f32()` en CMSIS-DSP) bajaría los lóbulos laterales a $-31{,}5\,\text{dB}$ y el error de amplitud máximo a 1,42 dB, a cambio de duplicar el ancho del lóbulo principal (sección 7.3).

4 Diseño de la aplicación

4.1 Arquitectura general

La figura 4 muestra las partes de la aplicación. La adquisición es la del Laboratorio 1: el CTIMER0 dispara cada conversión a través del INPUTMUX y el LPADC0 deja el resultado en su FIFO. La rutina de interrupción del ADC lo convierte a q15 y lo guarda en `buffer_in` hasta completar un bloque de N muestras; entonces levanta la bandera `buffer_listo`. El lazo principal ve la bandera, convierte el bloque a punto flotante en `buffer_out`, le aplica la FFT (o no, en *bypass*) y lo transmite por la UART al puerto COM virtual del MCU-Link, donde lo recibe la GUI de MATLAB. Las teclas y el touchpad sólo modifican la configuración: SW3 la frecuencia de muestreo (match del CTIMER0 y color del LED), SW2 el modo y el touchpad el tamaño N del próximo bloque.

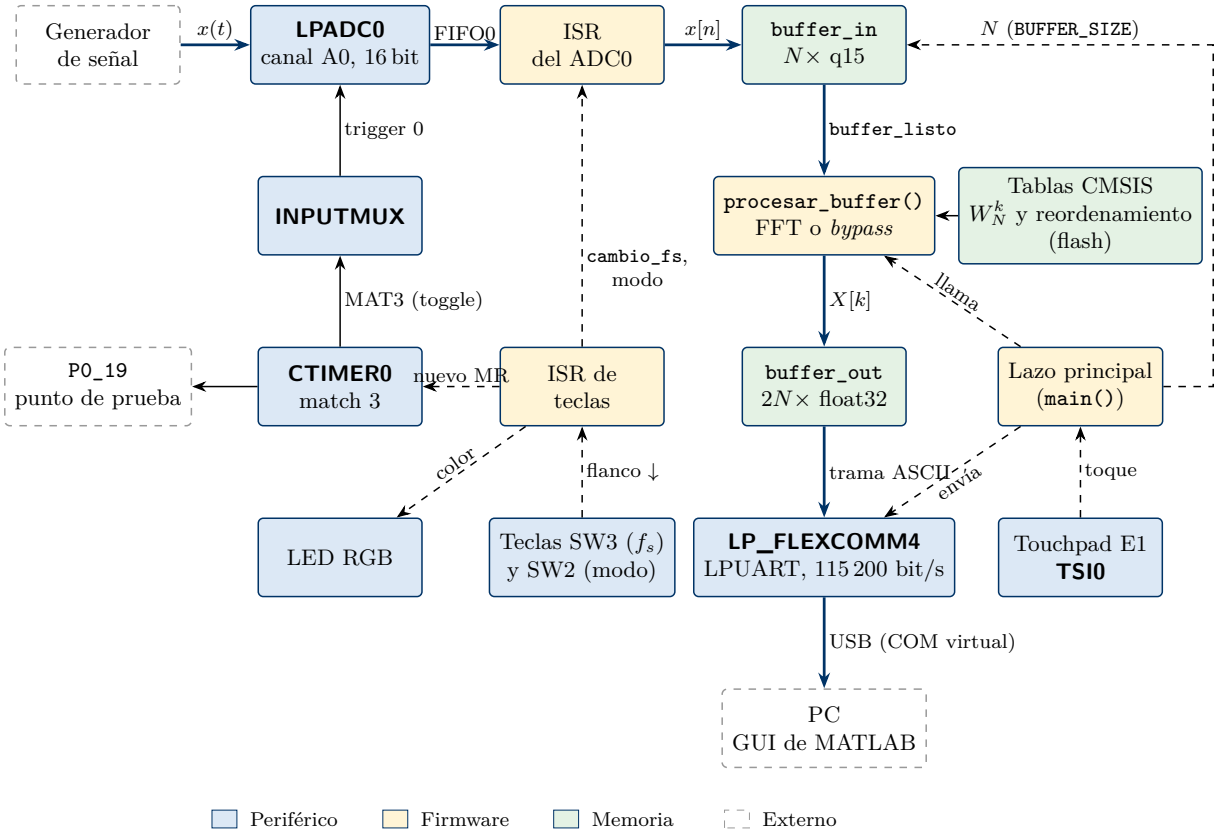


Figura 4: Diagrama de bloques de la aplicación. Trazo continuo azul: datos; negro: disparo y tablas; punteado: control.

4.2 Velocidades de muestreo

La generación de las velocidades no cambió respecto del Laboratorio 1: con el timer contando a 150 MHz, reiniciándose en el match y con la salida en toggle,

$$f_s = \frac{f_{\text{timer}}}{2\,(\text{MR} + 1)}. \tag{5}$$

La tabla 7 repite los valores de MR (arreglo `sample_vel`) y el color del LED de cada estado. La f_s real es la que el firmware envía en la cabecera de cada trama (sección 4.7), así que el eje de frecuencias de la GUI se construye con ella y no con la nominal.

Tabla 7: Velocidades de muestreo, valor de match programado y color del LED.

Estado	f_s nominal	MR	f_s real	Error	LED
E0	8 000 Hz	9 374	8 000,00 Hz	0,000 %	Verde
E1	16 000 Hz	4 687	15 998,29 Hz	−0,011 %	Amarillo
E2	22 000 Hz	3 408	22 000,59 Hz	0,003 %	Azul
E3	44 000 Hz	1 704	43 988,27 Hz	−0,027 %	Rojo
E4	48 000 Hz	1 562	47 984,64 Hz	−0,032 %	Blanco

4.3 Resolución y duración de la ventana

Con las cinco velocidades y los tres tamaños hay 15 combinaciones. La tabla 8 aplica la ecuación (2) a cada una, con la f_s real.

Tabla 8: Resolución en frecuencia $\Delta f = f_s/N$ y duración de la ventana $T_v = N/f_s$ para cada combinación.

	Δf [Hz]			T_v [ms]		
f_s	**512**	**1024**	**2048**	**512**	**1024**	**2048**
8 000 Hz	15,62	7,81	3,91	64	128	256
16 000 Hz	31,25	15,62	7,81	32	64	128
22 000 Hz	42,97	21,48	10,74	23,3	46,5	93,1
44 000 Hz	85,91	42,96	21,48	11,6	23,3	46,6
48 000 Hz	93,72	46,86	23,43	10,7	21,3	42,7

Se observan tres cosas:

- A N fijo, aumentar f_s amplía el rango analizado (hasta $f_s/2$) pero empeora la resolución en la misma proporción: a 48 kS/s y $N = 512$ los coeficientes están separados casi 94 Hz, seis veces más que a 8 kS/s.
- A f_s fija, duplicar N duplica la duración de la ventana y divide por dos Δf. También duplica la memoria, algo más que duplica el cálculo (tabla 4) y duplica el tiempo de transmisión (sección 4.8).
- Combinaciones distintas con el mismo cociente f_s/N dan la misma resolución: 8 kS/s con 512 muestras y 16 kS/s con 1024 tienen ambas $\Delta f \approx 15{,}6$ Hz y una ventana de 64 ms; la segunda cubre el doble de ancho de banda con el doble de coeficientes.

4.4 Máquina de estados

La configuración se modela con tres variables de estado independientes (figura 5):

- `state_counter`: índice de velocidad (E0 a E4). Avanza en forma circular con SW3; cada cambio reprograma el match del CTIMER0, cambia el color del LED y descarta el bloque en curso.
- `modo_seleccionado`: *bypass* (M0) o FFT (M1). Alterna con SW2.
- `BUFFER_SIZE`: tamaño de la ventana (N0 a N2: 512, 1024 y 2048). Avanza en forma circular con cada toque del touchpad.

Las tres actúan en cualquier estado. Al reiniciar la placa el sistema arranca en E0 (8 kS/s), en *bypass* y con $N = 512$. Ninguna de las tres se aplica a mitad de un bloque: la rutina del ADC las copia al comenzar cada bloque (sección 4.5).

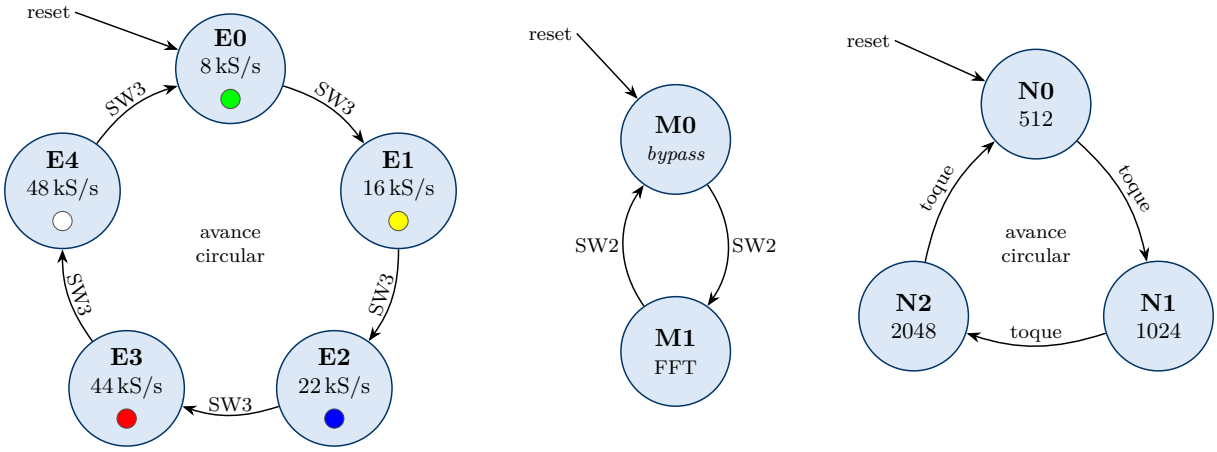


Figura 5: Izquierda: selección circular de la velocidad de muestreo con SW3 y color del LED en cada estado. Centro: modo, alternado con SW2. Derecha: tamaño de la ventana, recorrido con el touchpad E1.

4.5 Procesamiento por bloques

En el Laboratorio 2 cada muestra se procesaba en la misma interrupción en que llegaba. La FFT, en cambio, necesita las N muestras de la ventana antes de calcular cualquier coeficiente, así que el procesamiento es por bloques y se divide entre dos contextos que se pasan el buffer con una bandera, `buffer_listo` (figura 6):

- **Rutina del ADC (productor).** Mientras `buffer_listo` es falso, guarda cada muestra en `buffer_in[i]` e incrementa i. Al llegar a N pone i en cero y `buffer_listo` en 1. Mientras `buffer_listo` es verdadero, sigue leyendo la FIFO (para que no se desborde) pero descarta las muestras: el bloque no se toca.
- **Lazo principal (consumidor).** Cuando ve `buffer_listo` en 1, procesa y transmite el bloque, y recién al terminar la pone en 0, lo que habilita la adquisición del siguiente.

Como sólo un contexto escribe el buffer en cada momento, no hace falta duplicarlo ni deshabilitar interrupciones. El costo es que la adquisición queda en pausa durante el procesamiento y la transmisión: cada espectro es una *instantánea* de T_v segundos, y entre dos instantáneas se pierde el tiempo de transmisión.

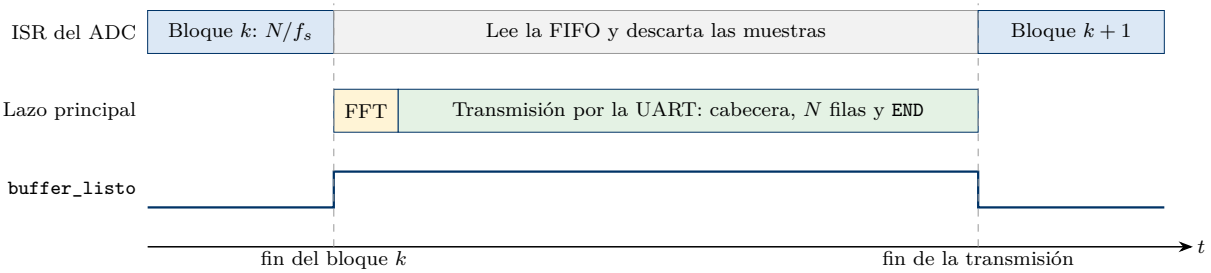


Figura 6: Línea de tiempo de un bloque (sin escala: la transmisión dura entre 16 y casi 100 veces más que la adquisición, tabla 10).

Configuración de cada bloque. El tamaño, el modo y la f_s se pueden cambiar en cualquier momento, incluso mientras se transmite. Para que el bloque sea coherente, la rutina del ADC copia `state_counter`, `modo_seleccionado` y `BUFFER_SIZE` en `fs_bloque`, `modo_bloque` y `buffer_size_bloque` al guardar la primera muestra ($i = 0$), y desde ahí usa sólo las copias. El lazo principal también usa sólo las copias para procesar y armar la cabecera, así que la FFT, la cantidad de filas y la f_s informada corresponden siempre a las muestras del bloque, aunque el usuario haya cambiado la selección en el medio. Un cambio de modo o de tamaño se ve recién en el bloque siguiente.

Muestras consecutivas. La FFT supone que las N muestras son consecutivas y equiespaciadas a una única f_s. Dos situaciones romperían esa hipótesis, y en las dos la rutina del ADC descarta el bloque parcial (pone i en cero) y vacía la FIFO:

- **Cambio de** f_s: la rutina de SW3 pone `cambio_fs` en 1. Sin el descarte, un bloque podría tener una parte muestreada a una velocidad y otra a otra, y su espectro mezclaría dos escalas de frecuencia.
- **Pérdida de muestras**: si el LPADC señala desborde de la FIFO (`kLPADC_ResultFIFO0 OverflowFlag`) o un disparo que no pudo atender (`kLPADC_TriggerExceptionFlag`), faltan muestras y el bloque ya no es una ventana continua.

`cambio_fs` arranca en 1, así que la primera interrupción también vacía la FIFO de cualquier conversión previa a la inicialización.

4.6 Formato de los datos

El LPADC0 trabaja en 16 bit, igual que en el Laboratorio 2, y la rutina del ADC centra el resultado restando la mitad de escala:

$$x_{q15}[n] = v - 32768 \in [-32768,\ 32767], \tag{6}$$

que es directamente un `q15_t`. El lazo principal lo pasa a punto flotante normalizado al copiarlo a `buffer_out`, con la parte imaginaria en cero:

$$\mathrm{Re}\,\{x[n]\} = \frac{x_{q15}[n]}{32768} \in [-1,\ 1), \qquad \mathrm{Im}\,\{x[n]\} = 0. \tag{7}$$

Así la amplitud 1 corresponde a la mitad del rango del ADC, y la amplitud de la ecuación (4) se lee en las mismas unidades.

Se eligió punto flotante en lugar de `arm_cfft_q15()` porque el Cortex-M33 del MCXN947 tiene FPU de simple precisión (el proyecto compila con `-mfpu=fpv5-sp-d16`, ABI *hard*) y porque la FFT en punto fijo tiene que escalar: como cada etapa puede duplicar la magnitud, `arm_cfft_q15()` divide por 2 en las etapas para no desbordar y entrega el resultado dividido por N, perdiendo $\log_2 N$ bits (11 bits de 15 para $N = 2048$). Con `float32_t`, de 24 bits de mantisa y exponente propio, el resultado sin normalizar (hasta $N/2 = 1024$ veces la amplitud de entrada) se representa sin pérdida apreciable y el piso de ruido queda determinado por el ADC, no por la aritmética.

4.7 Protocolo de comunicación

Cada bloque se envía como una trama de texto de $N + 2$ líneas terminadas en `\r\n` (tabla 9).

Tabla 9: Formato de una trama.

Línea	Formato	Contenido
Cabecera	`TP3,<seq>,<modo>,<N>,<fs_mHz>`	Número de trama, modo (0: *bypass*, 1: FFT), tamaño N y f_s real del bloque en milihertz
Filas $k = 0, \ldots, N-1$	`<k>,<RE>,<IM>`	Índice y bits IEEE-754 de la parte real e imaginaria, en hexadecimal de 8 dígitos
Fin	`END`	Fin de la trama

Por ejemplo, una trama FFT de 512 puntos a 8 kS/s empieza y termina así (los valores de las filas son ilustrativos):

Código 1: Ejemplo de trama.

```
TP3,17,1,512,8000000
0,3C4985F0,00000000
1,BE800000,3F000000
...
64,43800000,C1480000
...
511,BE800000,BF000000
END
```

Las decisiones del formato son:

- **La f_s viaja en la cabecera**, calculada con la ecuación (5) a partir del MR del bloque y en milihertz, para que sea un entero exacto con tres decimales (15 998 293 mHz en E1). El cálculo se hace en 64 bits, porque $1000 \cdot f_{\text{timer}} = 1{,}5 \times 10^{11}$ no entra en 32. La GUI no necesita conocer la tabla de velocidades.
- **Los valores se envían como los bits del `float` en hexadecimal**, no en decimal. El proyecto usa la biblioteca Redlib con `printf` sólo de enteros (`CR_INTEGER_PRINTF`, `PRINTF_FLOAT_ENABLE=0`), así que `snprintf` no puede formatear flotantes; copiar los bits a un `uint32_t` con `memcpy` e imprimirlo con `%08lX` no necesita soporte de punto flotante, transmite el valor exacto sin redondeo y da campos de ancho fijo, fáciles de validar. En MATLAB se recupera con `typecast(uint32(hex2dec(...)),'single')`.
- **Cada fila lleva su índice**, y la trama tiene cabecera y cierre. La GUI verifica que los índices sean consecutivos y que la trama termine con `END`; si una línea se pierde o llega corrupta, descarta la trama y se vuelve a sincronizar con la siguiente cabecera.
- **Se envía el espectro completo** (N coeficientes, incluida la mitad conjugada), que es exactamente el contenido de `buffer_out`. En *bypass* el mismo formato lleva las N muestras de la ventana, con la parte imaginaria en cero.

La UART se escribe con `LPUART_WriteBlocking()`, que espera a que haya lugar en la FIFO de transmisión. Como esa espera ocurre en el lazo principal y no en una interrupción, no afecta a la adquisición.

4.8 Restricción de tiempo

A diferencia del Laboratorio 2, aquí no hay que terminar el cálculo antes de la muestra siguiente: el bloque está protegido por `buffer_listo` y el lazo principal puede tardar lo que necesite. Lo

que se ve afectado es el período con que se refresca el espectro,

$$T_{\text{bloque}} = T_v + T_{\text{FFT}} + T_{\text{tx}}, \qquad T_{\text{tx}} = \frac{\text{bytes de la trama} \cdot 10}{115\,200}, \tag{8}$$

donde cada byte ocupa 10 bits en la línea (inicio, 8 datos y parada). Una fila tiene entre 21 y 24 bytes según la cantidad de dígitos del índice. La tabla 10 aplica la ecuación (8) a la menor y a la mayor f_s, sin T_{FFT}.

Tabla 10: Bytes por trama, tiempo de transmisión a 115 200 bit/s, duración de la ventana y período de refresco del espectro.

			T_v [ms]		T_{bloque} [s]	
N	**Bytes**	T_{tx} [s]	8 kS/s	48 kS/s	8 kS/s	48 kS/s
512	11 693	1,02	64	10,7	1,08	1,03
1 024	23 494	2,04	128	21,3	2,17	2,06
2 048	48 070	4,17	256	42,7	4,43	4,22

La transmisión domina: dura entre 16 y casi 100 veces la ventana, y el espectro se refresca aproximadamente una vez por segundo con $N = 512$ y una vez cada cuatro segundos con $N = 2048$, casi sin importar la f_s.

El cálculo de la FFT es mucho más corto. Con la estimación habitual de $5\,N \log_2 N$ operaciones de punto flotante (cada mariposa radix-2: un producto complejo, cuatro productos y dos sumas, más dos sumas complejas), la FFT de 2 048 puntos son unas 112 640 operaciones, del orden de 1 ms a 150 MHz aun contando los accesos a memoria y la compilación sin optimizar.

¿Se podría transmitir en forma continua? Para no perder muestras, cada trama tendría que enviarse en menos tiempo que T_v, es decir, a una tasa de (bytes por coeficiente) $\cdot\, f_s$. Con el formato actual (unos 23,5 bytes por coeficiente) eso es 1,9 Mbit/s a 8 kS/s y 11,3 Mbit/s a 48 kS/s. Incluso con el formato más compacto posible, en binario y sólo con los $N/2$ coeficientes no redundantes (8 bytes cada uno, 4 bytes por muestra de entrada), harían falta 320 kbit/s y 1,92 Mbit/s. Las dos cifras superan los 115 200 bit/s del enlace, así que el esquema de instantáneas no es una simplificación sino la única opción con este enlace. Para un análisis continuo habría que reducir lo que se transmite (por ejemplo, sólo el módulo de unos pocos coeficientes) o promediar varios espectros en la placa antes de enviar uno.

4.9 Uso de memoria

`buffer_in` y `buffer_out` se dimensionan una sola vez para el máximo, `BUFFER_MAX_SIZE` = 2 048, y los bloques más chicos usan sólo su parte inicial. `buffer_in` ocupa 4 KiB (2048×2 bytes) y `buffer_out` 16 KiB ($2 \times 2048 \times 4$ bytes), unos 20 KiB de RAM en total. La FFT trabaja en el lugar sobre `buffer_out`, sin buffers auxiliares, y las tablas de la tabla 5 están en flash. Que la entrada y la salida sean buffers distintos es lo que pide la consigna, y además permite conservar las muestras originales en `buffer_in` aunque la FFT sobrescriba `buffer_out`.

5 Implementación

La lógica propia está en `source/TP3.c`; el resto del proyecto es código generado por Config Tools, drivers del SDK y CMSIS-DSP. Los scripts de MATLAB están en `TP3/matlab/`. El código completo está en el Anexo A.

5.1 Constantes y variables globales

Los valores de match se mantienen en `sample_vel`, igual que en los laboratorios anteriores. `BUFFER_MAX_SIZE` (2 048) fija la capacidad de los buffers. `BUFFER_SIZE` conserva el nombre de la constante del Laboratorio 2 (`#define BUFFER_SIZE 512`), pero ahora es una variable `volatile` que modifica el touchpad. Las variables que comparten las interrupciones y el lazo principal (`state_counter`, `modo_seleccionado`, `BUFFER_SIZE`, `buffer_listo` y `cambio_fs`) se declaran `volatile`. `fs_bloque`, `modo_bloque` y `buffer_size_bloque` son la configuración del bloque (sección 4.5): la escribe la rutina del ADC al comenzar un bloque y la lee el lazo principal sólo mientras `buffer_listo` es verdadero. `pte_fft` apunta a la instancia de CMSIS-DSP del tamaño activo. El estado del touchpad (`touch_base`, `touch_tocado` y los demás `touch_*`) y la base de tiempo `ms_ticks` son los del Laboratorio 2.

5.2 Programa principal

`main()` inicializa pines y relojes, enciende el VREF (sección 2.4) y recién después inicializa los periféricos generados por Config Tools (CTIMER0, LPADC0, GPIO0 y la UART). A continuación fija el color del LED para E0, configura el SysTick para interrumpir cada 1 ms, mide la base del touchpad (`init_touch()`), envía la línea `TP3 boot` (que la GUI ignora) y entra en el lazo principal (figura 7).

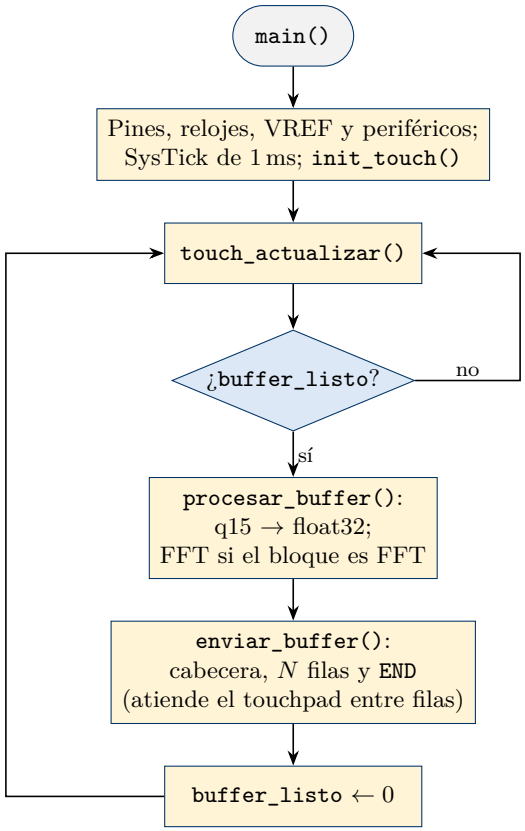


Figura 7: Lazo principal: touchpad, procesamiento y transmisión.

5.3 Rutina del ADC

El código 2 muestra el núcleo de la rutina del ADC y la figura 8 su diagrama de flujo. Primero se atiende el descarte por cambio de f_s o pérdida de muestras; si no corresponde, se lee la conversión, se centra a q15 y, si el bloque no está protegido, se guarda.

Código 2: Adquisición de un bloque en `ADC0_IRQHANDLER()` (`source/TP3.c`).

```
if (cambio_fs ||
    (status_flag & (kLPADC_ResultFIFO0OverflowFlag |
                    kLPADC_TriggerExceptionFlag))) {
    index_buffer = 0;                /* descarta el bloque parcial */
    cambio_fs = false;
    while (LPADC_GetConvResult(ADC0_PERIPHERAL, &result, 0)) {
    }                                /* vacia la FIFO */

} else if (LPADC_GetConvResult(ADC0_PERIPHERAL, &result, 0)) {
    q15_t muestra = (q15_t)((int32_t)result.convValue - 32768);

    if (!buffer_listo) {
        if (index_buffer == 0) {     /* configuracion de ESTE bloque */
            fs_bloque = state_counter;
            modo_bloque = modo_seleccionado;
            buffer_size_bloque = BUFFER_SIZE;
        }
        buffer_in[index_buffer] = muestra;
        index_buffer++;
        if (index_buffer == buffer_size_bloque) {
            index_buffer = 0;
            buffer_listo = true;     /* entrega el bloque a main */
        }
```

```
24      }
25  }
```

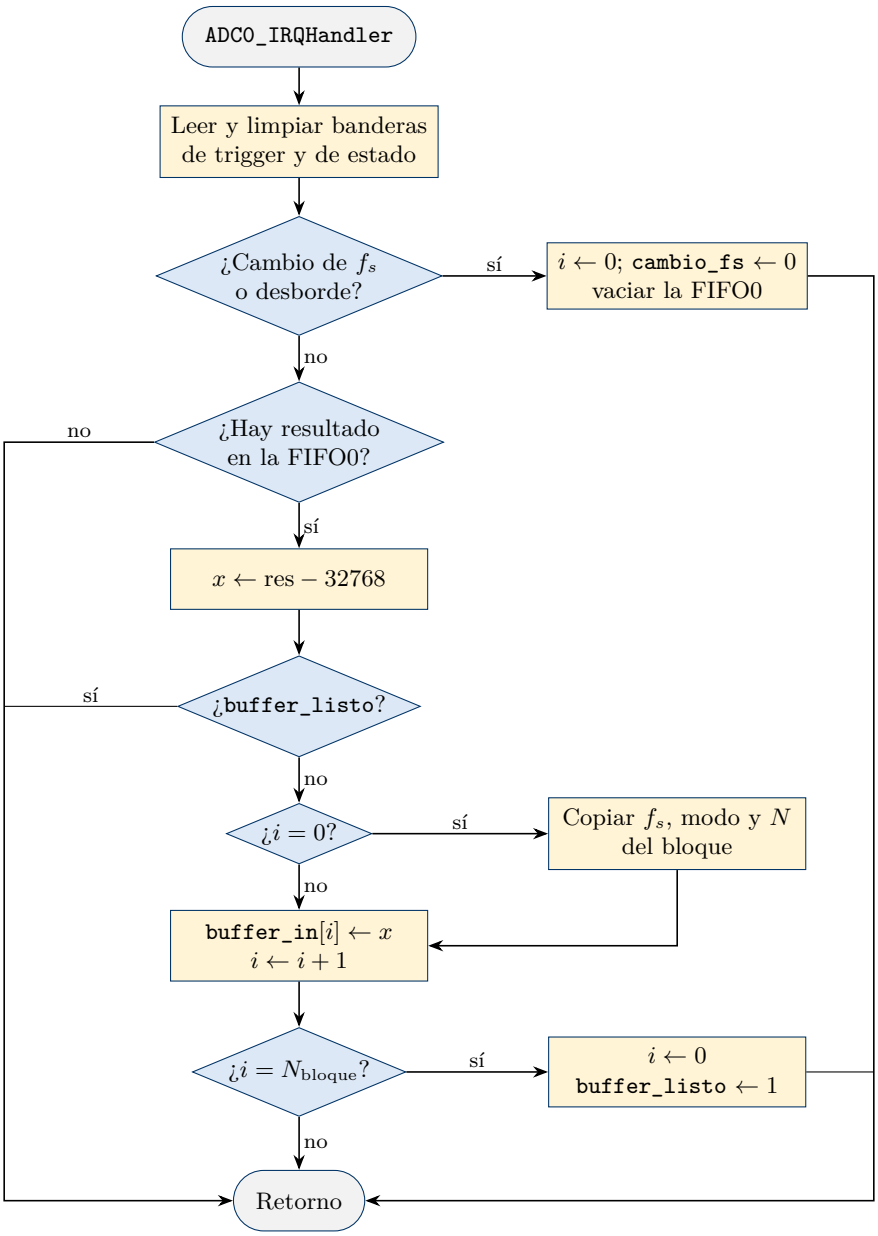


Figura 8: Diagrama de flujo de la rutina de fin de conversión del ADC. Con `buffer_listo` en 1 la muestra se lee (la FIFO no se llena) pero se descarta.

5.4 Procesamiento del bloque

`procesar_buffer()` (código 3) convierte el bloque a complejo en punto flotante y, si el bloque se adquirió en modo FFT, elige la instancia de CMSIS-DSP del tamaño del bloque y calcula la transformada en el lugar. `init_fft()` conserva el nombre de la versión inicial, en la que el tamaño era una constante de compilación y la instancia se elegía una sola vez con `#if`; ahora se llama en cada bloque y elige con `buffer_size_bloque`, el tamaño con que se adquirió, y no con `BUFFER_SIZE`, que el touchpad puede haber cambiado mientras tanto. Elegir la instancia no cuesta nada: es asignar un puntero a una estructura constante.

Código 3: Selección de la instancia y cálculo de la FFT (`source/TP3.c`).

```
void init_fft(void) {
    switch (buffer_size_bloque) {
        case 512U:  pte_fft = &arm_cfft_sR_f32_len512;  break;
        case 1024U: pte_fft = &arm_cfft_sR_f32_len1024; break;
        case 2048U: pte_fft = &arm_cfft_sR_f32_len2048; break;
    }
}

void procesar_buffer(void) {
    for (uint16_t i = 0; i < buffer_size_bloque; i++) {
        buffer_out[2*i]   = (float32_t)buffer_in[i] / 32768.0f;  /* Re */
        buffer_out[2*i+1] = 0.0f;                                 /* Im */
    }
    if (modo_bloque == MODO_FFT) {
        init_fft();
        arm_cfft_f32(pte_fft, buffer_out, 0, 1);  /* directa, orden natural */
    }
}
```

En *bypass* la función sólo hace la copia: el buffer de salida es distinto del de entrada, como pide la consigna, y contiene las muestras sin transformar.

5.5 Transmisión

`enviar_buffer()` (código 4) arma la cabecera con la configuración del bloque y envía una fila por coeficiente. Entre fila y fila llama a `touch_actualizar()`: la transmisión de un bloque de 2 048 puntos tarda más de cuatro segundos (tabla 10), y sin esa llamada el touchpad no respondería durante ese tiempo. Como el touchpad sólo cambia `BUFFER_SIZE`, la trama en curso no se altera.

Código 4: Armado y envío de la trama en `enviar_buffer()` (`source/TP3.c`).

```
uint32_t fs_millihz =
    (uint32_t)((1000ULL * CTIMER0_TICK_FREQ) /
    (2ULL * (CTIMER0_config.prescale + 1ULL) *
     (sample_vel[fs_bloque] + 1ULL)));

snprintf(linea, sizeof(linea), "TP3,%lu,%u,%u,%lu\r\n",
         (unsigned long)++numero_bloque, (unsigned)modo_bloque,
         (unsigned)buffer_size_bloque, (unsigned long)fs_millihz);
if (!enviar_texto_uart(linea)) return;

for (uint16_t i = 0; i < buffer_size_bloque; i++) {
    uint32_t real, imag;
    memcpy(&real, &buffer_out[2*i],   sizeof(real));  /* bits del float */
    memcpy(&imag, &buffer_out[2*i+1], sizeof(imag));
    snprintf(linea, sizeof(linea), "%u,%08lX,%08lX\r\n", (unsigned)i,
             (unsigned long)real, (unsigned long)imag);
    if (!enviar_texto_uart(linea)) return;
    touch_actualizar(ms_ticks);     /* el touchpad no espera a END */
}
(void)enviar_texto_uart("END\r\n");
```

Se usa `memcpy` en lugar de un *cast* de punteros (`*(uint32_t *)&f`) porque reinterpretar un `float` a través de un puntero a entero viola las reglas de *aliasing* de C y el compilador podría optimizarlo de forma incorrecta; `memcpy` es la forma portable y, para 4 bytes, el compilador la reduce a una sola transferencia.

5.6 Rutinas de las teclas

La rutina de SW3 avanza `state_counter` en forma circular, reprograma el CTIMER0 con `setup_new_match()` (detiene el timer, carga el nuevo match, lo reinicia y lo arranca, como en el Laboratorio 1), actualiza el color del LED con `set_state()` y pone `cambio_fs` en 1. La rutina de SW2 sólo alterna `modo_seleccionado`. Ninguna toca los buffers: el efecto se aplica en la rutina del ADC al comenzar el bloque siguiente (figura 9).

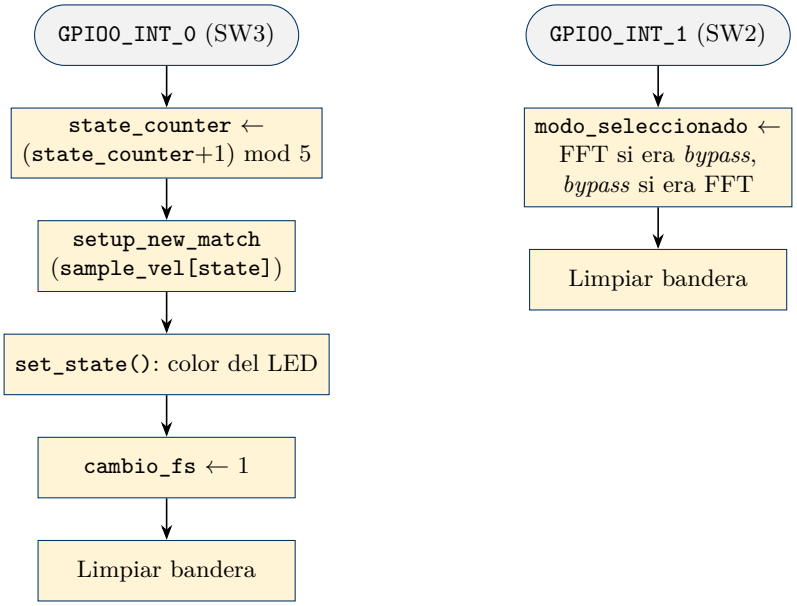


Figura 9: Diagramas de flujo de las rutinas de las teclas. Izquierda: cambio de velocidad (SW3). Derecha: cambio de modo (SW2).

5.7 Touchpad

El touchpad se lee exactamente como en el Laboratorio 2, desde el lazo principal y sin interrupciones: al arrancar, `init_touch()` configura el TSI0 en modo *self-cap* con disparo por software y mide la base como el promedio de 16 barridos sin tocar (por eso no hay que tocar el touchpad durante el reset). Después, `touch_actualizar()` dispara un barrido cada 10 ms y consulta si terminó en las vueltas siguientes, sin esperar; si un barrido no termina en 50 ms, el touchpad se desactiva y el resto del programa sigue funcionando. Una lectura cuenta como toque si está más de 20 cuentas por debajo de la base y como liberación si vuelve a quedar a menos de 10 (histéresis), y el cambio se acepta con 3 lecturas seguidas (antirrebote de unos 30 ms). Sólo el paso de *sin tocar* a *tocado* cuenta como toque, así que mantener el dedo apoyado no avanza varios tamaños.

Lo único que cambió es la acción: cada toque nuevo avanza `BUFFER_SIZE` en forma circular, 512 → 1024 → 2048 → 512. El valor se aplica al comenzar el próximo bloque. En el TP3 se quitó el aviso por `PRINTF` cuando el touchpad no responde, porque la UART transporta el protocolo y un texto intercalado rompería una trama.

5.8 Aplicación de MATLAB

En `TP3/matlab/` hay tres scripts:

- `recibir_tp3.m(puerto, baud)` es la GUI. Abre el puerto con `serialport` (115 200 bit/s por defecto) y terminador `CR/LF`, y en un lazo que dura mientras la ventana esté abierta:

busca una línea que empiece con `TP3,`, valida la cabecera ($N \in \{512, 1024, 2048\}$, modo 0 o 1, $f_s > 0$), lee las N filas verificando con una expresión regular el formato y que el índice sea el esperado, decodifica los flotantes y comprueba el `END`. En modo FFT dibuja el espectro de un solo lado de la ecuación (4) entre 0 y $f_s/2$, con el eje en hertz construido con la f_s de la cabecera; en *bypass* dibuja las muestras contra el tiempo. El título muestra el número de trama, el modo, N y f_s. Una trama con errores genera una advertencia y se descarta, sin cerrar la GUI.

- `diagnosticar_tp3.m` captura una sola trama, la guarda en `.mat` y `.txt`, e informa el rango de la parte real y la imaginaria, la cantidad de valores no nulos y no finitos, los primeros datos en hexadecimal y en decimal y, en modo FFT, la componente continua y el pico principal (frecuencia y amplitud). Se usó para depurar el enlace y para tomar los valores numéricos de la sección 6.

- `explorar_fft_tp3.m` es una referencia en la PC: genera una senoidal, la cuantiza a q15 como el ADC, calcula su FFT con `fft` y la grafica con la misma normalización que la GUI. Permite prever el resultado de cada combinación de f_s, N y f_0 (por ejemplo, la fuga de la sección 3.5).

Código 5: Decodificación de una fila y espectro de un solo lado en `recibir_tp3.m`.

```
tok = regexp(row,'^(\d+),([0-9A-F]{8}),([0-9A-F]{8})$','tokens','once');
if isempty(tok) || str2double(tok{1}) ~= k
    error('TP3:Frame','Trama incompleta o desordenada.');
end
re = double(typecast(uint32(hex2dec(tok{2})),'single'));
im = double(typecast(uint32(hex2dec(tok{3})),'single'));
X(k+1) = complex(re,im);
% ... al completar la trama, en modo FFT:
A = abs(X(1:N/2+1))/N;
A(2:end-1) = 2*A(2:end-1);
plot(ax,(0:N/2)*Fs/N,A);
```

6 Verificación

6.1 Procedimiento

1. Conectar el generador a `J4[2]` (ADC0_A0) con una senoidal de 1 kHz y un *offset* que la mantenga dentro del rango del ADC. Conectar la placa a la PC por el USB del MCU-Link.
2. Ejecutar `recibir_tp3("COMx", 115200)` en MATLAB (con ningún otro programa usando el puerto).
3. Con $f_s = 8\,\text{kS/s}$ y $N = 512$, verificar en *bypass* que la GUI muestra la senoidal en el tiempo; pasar a FFT con SW2 y verificar que el pico está en 1 kHz.
4. Recorrer los tres tamaños con el touchpad a f_s fija, y las cinco velocidades con SW3 a N fijo, verificando en el título de la GUI que N y f_s cambian en el bloque siguiente.
5. Repetir con una senoidal de 5 kHz a 8 kS/s y a 16 kS/s (aliasing) y con una cuadrada de 500 Hz (armónicos).
6. Para los valores numéricos, cerrar la GUI y capturar una trama con `diagnosticar_tp3`.

6.2 Verificación por simulación

Antes de medir en la placa se calculó en la PC, con `explorar_fft_tp3.m`, el espectro esperado para $f_s = 8\,\text{kS/s}$ y $N = 512$. Con $f_0 = 1\,000\,\text{Hz}$ el script informa una ventana de 64 ms, una separación de 15,625 Hz y un único pico en 1 000 Hz de amplitud 0,5; con $f_0 = 1\,005\,\text{Hz}$ aparece la fuga de la tabla 6.

Captura pendiente: **MATLAB**
guardar como `img/sim-fuga-1000.png`

Figura 10: MATLAB. `explorar_fft_tp3.m` con $f_0 = 1\,000\,\text{Hz}$, $f_s = 8\,\text{kS/s}$, $N = 512$: la senoidal tiene un número entero de ciclos y cae en un solo coeficiente.

Captura pendiente: **MATLAB**
guardar como `img/sim-fuga-1005.png`

Figura 11: MATLAB. `explorar_fft_tp3.m` con $f_0 = 1\,005\,\text{Hz}$: fuga espectral hacia los coeficientes vecinos.

6.3 Transmisión y modo *bypass*

Captura pendiente: **Terminal serie**
guardar como `img/terminal-trama.png`

Figura 12: Terminal serie. Comienzo de una trama recibida: cabecera, primeras filas.

Captura pendiente: **GUI de MATLAB**
guardar como `img/gui-bypass-8k.png`

Figura 13: GUI de MATLAB. Modo *bypass*: senoidal de 1 kHz muestreada a 8 kS/s, $N = 512$ (64 ms, 64 ciclos).

6.4 Espectro al variar el tamaño de la ventana

A $f_s = 8\,\text{kS/s}$, 1 kHz es múltiplo de Δf para los tres tamaños (coeficientes 64, 128 y 256), así que el pico debe quedar en 1 000 Hz con la amplitud de la senoidal.

Captura pendiente: **GUI de MATLAB**
guardar como `img/gui-fft-8k-512.png`

Figura 14: GUI de MATLAB. FFT de una senoidal de 1 kHz a 8 kS/s con $N = 512$ ($\Delta f = 15{,}6\,\text{Hz}$).

Captura pendiente: **GUI de MATLAB**
guardar como `img/gui-fft-8k-1024.png`

Figura 15: GUI de MATLAB. Igual que la figura 14 con $N = 1024$ ($\Delta f = 7{,}8\,\text{Hz}$).

Captura pendiente: **GUI de MATLAB**
guardar como `img/gui-fft-8k-2048.png`

Figura 16: GUI de MATLAB. Igual que la figura 14 con $N = 2048$ ($\Delta f = 3{,}9$ Hz).

6.5 Espectro al variar la frecuencia de muestreo

A $N = 512$ y velocidades mayores, 1 kHz ya no es múltiplo de Δf: el pico se desplaza al coeficiente más cercano y aparece fuga. La tabla 11 incluye el coeficiente esperado.

Captura pendiente: **GUI de MATLAB**
guardar como `img/gui-fft-22k-512.png`

Figura 17: GUI de MATLAB. FFT de una senoidal de 1 kHz a 22 kS/s con $N = 512$ ($\Delta f = 43$ Hz).

Captura pendiente: **GUI de MATLAB**
guardar como `img/gui-fft-48k-512.png`

Figura 18: GUI de MATLAB. FFT de una senoidal de 1 kHz a 48 kS/s con $N = 512$ ($\Delta f = 93{,}7$ Hz).

6.6 Aliasing y armónicos

El LPADC0 no tiene filtro antialias a la entrada, así que una componente por encima de $f_s/2$ aparece reflejada en $|f - mf_s|$. Una senoidal de 5 kHz muestreada a 8 kS/s debe verse en 3 kHz; a 16 kS/s, en su frecuencia real. Una cuadrada de 500 Hz y amplitud A tiene sólo armónicos impares, de amplitud $4A/(\pi m)$; a 8 kS/s los que superan 4 kHz se pliegan sobre la banda.

Captura pendiente: **GUI de MATLAB**
guardar como `img/gui-alias-8k.png`

Figura 19: GUI de MATLAB. Senoidal de 5 kHz a 8 kS/s: aparece en 3 kHz.

Captura pendiente: **GUI de MATLAB**
guardar como `img/gui-alias-16k.png`

Figura 20: GUI de MATLAB. La misma senoidal a 16 kS/s: aparece en 5 kHz.

Captura pendiente: **GUI de MATLAB**
guardar como `img/gui-cuadrada-8k.png`

Figura 21: GUI de MATLAB. Cuadrada de 500 Hz a 8 kS/s: armónicos impares y armónicos plegados.

6.7 Valores medidos

Tabla 11: Pico principal esperado y medido (con `diagnosticar_tp3.m`). El esperado es el coeficiente más cercano a la frecuencia de entrada.

f_s [Hz]	N	Δf [Hz]	f_{in} [Hz]	Pico esperado [Hz]	Pico medido [Hz]	Amplitud medida
8 000	512	15,63	1 000	1 000	[]	[]
8 000	1024	7,81	1 000	1 000	[]	[]
8 000	2048	3,91	1 000	1 000	[]	[]
22 000,59	512	42,97	1 000	988,31	[]	[]
47 984,64	512	93,72	1 000	1 030,92	[]	[]
8 000	512	15,63	5 000	3 000	[]	[]
15 998,29	512	31,25	5 000	4 999,47	[]	[]

6.8 Cumplimiento de los requerimientos

Tabla 12: Verificación de los requerimientos de la consigna.

Req.	Cómo se implementa	Evidencia	Resultado
R1	Cadena CTIMER0 + INPUTMUX + LPADC0 del Laboratorio 1; SW3 elige la f_s; cambio de f_s con descarte del bloque	Figuras 14, 18	✓
R2	`buffer_in[2048]`; el touchpad elige 512, 1024 o 2048, aplicado al comenzar el bloque	Figuras 14 a 16	[verificar]
R3	`arm_cfft_f32()` con la instancia del tamaño del bloque	Tabla 11	✓
R4	SW2 alterna FFT y *bypass*; el resultado va a `buffer_out`, distinto de `buffer_in`	Figuras 13 y 14	✓
R5	LPUART (LP_FLEXCOMM4) a 115 200 bit/s, trama con cabecera, N filas y `END`	Figura 12	✓
R6	`recibir_tp3.m`: espectro de un solo lado en hertz o señal en el tiempo	Figuras 13 a 21	✓
R7	Análisis de resolución, tiempos y fuga	Secciones 4.3, 4.8 y 7	✓

7 Conclusiones

Se implementó sobre la FRDM-MCXN947 una aplicación que adquiere la señal con la cadena del Laboratorio 1 a cinco velocidades de muestreo, la guarda en bloques de 512, 1024 o 2048 muestras, les aplica la FFT con `arm_cfft_f32()` de CMSIS-DSP (o las deja pasar sin transformar, en *bypass*) hacia un buffer de salida distinto del de entrada, y envía el resultado por la UART a una GUI de MATLAB que dibuja el espectro o la señal. SW3 elige la frecuencia de muestreo, SW2 el modo y el touchpad el tamaño de la ventana, sin recompilar.

7.1 Efecto de la frecuencia de muestreo y del tamaño de la ventana

- **La resolución depende sólo de la duración de la ventana.** $\Delta f = f_s/N = 1/T_v$: para distinguir dos componentes separadas δf hace falta observar la señal durante al menos $1/\delta f$ segundos, y eso se logra con más muestras o con menor f_s. Subir f_s con N fijo amplía el ancho de banda analizado ($f_s/2$) pero acorta la ventana y empeora la resolución en la misma proporción (de 15,6 Hz a 93,7 Hz al pasar de 8 kS/s a 48 kS/s con $N = 512$). Para cada señal conviene la menor f_s que cumpla con Nyquist y el mayor N que se pueda pagar.
- **La grilla de frecuencias se mueve con f_s y con N.** Una misma senoidal puede caer justo en un coeficiente con una combinación (1 kHz a 8 kS/s) y entre dos con otra (a 22 kS/s o 48 kS/s), con un error de amplitud de hasta 3,92 dB y fuga hacia los coeficientes vecinos. Duplicar N achica ese error de frecuencia a la mitad pero no lo elimina; para medir amplitudes con precisión hay que aplicar una ventana.
- **Una ventana más larga baja el piso de ruido por coeficiente.** El ruido de cuantización se reparte entre los N coeficientes mientras que la energía de una senoidal se concentra en uno, así que cada vez que se duplica N el piso de ruido baja 3 dB respecto del tono y se pueden ver componentes más débiles.
- **Una ventana más larga cuesta más en todo lo demás.** La memoria crece con N, el cálculo con $N \log_2 N$, el tiempo de transmisión con N y la latencia (el tiempo hasta tener el primer coeficiente) con T_v. Además, una ventana larga promedia la señal durante más tiempo: si la señal cambia dentro de la ventana, el espectro mezcla los distintos estados. Es el compromiso entre resolución en frecuencia y resolución temporal.
- **La frecuencia de muestreo fija también el aliasing.** Sin filtro antialias, toda componente por encima de $f_s/2$ se pliega dentro de la banda y es indistinguible de una componente real; subir f_s es la única defensa en este sistema.
- **En esta implementación el costo de la FFT no es el cuello de botella.** El cálculo dura del orden de un milisegundo, mientras que la trama tarda entre uno y cuatro segundos en salir por la UART. El período de refresco depende casi sólo de N y casi nada de f_s (tabla 10).

[Comentar lo observado en las figuras 14 a 21 y en la tabla 11: ancho del pico y piso de ruido al aumentar N, desplazamiento del pico y fuga al aumentar f_s, frecuencia aparente de la senoidal de 5 kHz y amplitud de los armónicos de la cuadrada.]

7.2 Decisiones de diseño

- **Procesamiento por bloques con una bandera de entrega**: la rutina del ADC llena el buffer y el lazo principal lo procesa, sin duplicar buffers ni deshabilitar interrupciones.

- **Configuración copiada al comenzar cada bloque**: la FFT, la cantidad de filas y la f_s informada corresponden siempre a las muestras, aunque el usuario cambie la selección en el medio.
- **Descarte del bloque parcial** al cambiar f_s o al perder muestras, para que cada ventana sea continua y uniforme.
- **FFT en punto flotante** con las instancias precalculadas de CMSIS-DSP, aprovechando la FPU y evitando el escalado de la FFT en punto fijo.
- **Trama de texto con los bits IEEE-754 en hexadecimal**: exacta, sin soporte de `printf` de flotantes, con índice y cierre para detectar errores, y con la f_s real en la cabecera.
- **Touchpad atendido durante la transmisión**, para que siga respondiendo aunque una trama tarde varios segundos.
- **VREF encendido explícitamente** antes de inicializar el ADC, ya que al quitar el DAC dejó de encenderse como efecto lateral.

7.3 Mejoras posibles

- **Transmitir menos**: sólo los $N/2+1$ coeficientes no redundantes, en binario y a una velocidad mayor (el COM virtual del MCU-Link admite velocidades mayores que 115 200 bit/s), reduciría el tiempo de transmisión más de diez veces.
- **Usar `arm_rfft_fast_f32()`**, que aprovecha que la entrada es real y calcula la mitad de operaciones con la mitad de memoria.
- **Aplicar una ventana** (Hann o Hamming, disponibles en CMSIS-DSP) antes de la FFT para bajar la fuga y el error de amplitud, a cambio de un lóbulo principal más ancho.
- **Adquisición continua con doble buffer y DMA**, para no perder muestras durante el procesamiento si el enlace lo permitiera, o para promediar varios espectros en la placa antes de enviar uno.
- **Antirrebote de las teclas**: un rebote de SW2 puede alternar el modo dos veces y dejarlo como estaba.
- **Barreras de memoria explícitas**: la entrega del bloque depende del orden entre la escritura de `buffer_in` (y de la configuración del bloque) y la de `buffer_listo`. Funciona porque el núcleo es único y `buffer_listo` es `volatile`, pero declarar también `volatile` la configuración del bloque o agregar `__DMB()` haría el código robusto frente a optimizaciones más agresivas del compilador.

A Código fuente

El proyecto completo de MCUXpresso (incluidos los archivos generados por Config Tools, los drivers del SDK y CMSIS-DSP) está en la carpeta `TP3/` del repositorio, y los scripts de MATLAB en `TP3/matlab/`. El repositorio es:

`https://github.com/Mati-Costamagna/procesamiento-digital-de-senales`